Talk transcript: Building quantum codes with transversal CCZ

Virginia Tech Applied Algebra Seminar, October 22, 2026. Joint work with Ken Brown. About 41 minutes, then discussion.

How to use this transcript

Each section is one frame of the deck, in order. The heading gives the displayed slide number. The five unnumbered outline frames between parts have their own sections.

- **Clock** is the planned start and end time. **Budget** is the time for the frame. **Clicks** is the number of overlay steps after the first view.
- **[click]** marks where to advance. The text after it describes what has just appeared.
- *(pause)* means stop for about two seconds and let the slide be read.
- *(ask, then wait 7 seconds)* marks a question to the audience. A fallback answer follows in case nobody answers.
- Spoken text is written at about 135 words per minute. Read it aloud at a steady pace with the clock running, and mark where you run over.
- **Background (not spoken)** boxes on slides 2 to 6 collect the facts behind each sentence, with short answers to likely questions. Read them before rehearsing, not during.

Terms and notation follow the standard literature and our paper: Pauli strings $X(a)$ and $Z(b)$, stabilizers and stabilizer generators, X-type and Z-type stabilizers, logical operators, the CSS code of $\mathcal{C}_2 \subseteq \mathcal{C}_1$, quasi-transversal, triorthogonal, column indicator. Two terms come from our paper and are defined on the slides: the signed weight and the logical label.

Winston's "How to Speak" applied to this talk

These heuristics are from Patrick Winston's MIT lecture "How to Speak" (MIT OpenCourseWare RES.TLL-005, IAP 2018). Each one is built into the script.

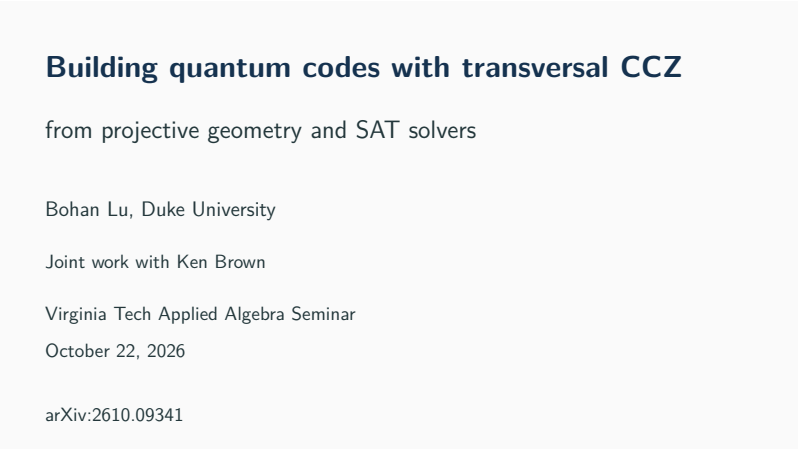
- **Empowerment promise, not a joke.** Slide 1 tells the audience what they will be able to do at the end, before 1:00. The result itself waits for slide 17, after the background that lets the audience follow it. Winston says a speaker should show both a vision and that something has been done within five minutes. The promise is the vision, and the eight-qubit cube on slides 7 to 12 shows that something has been done.
- **Cycling.** The three steps of the construction are said at slides 17, 18 and 31: "Geometry fixes the stabilizers S, a search finds the logical rows K, and linear algebra over $\mathbb{Z}_4$ finds the signs."
- **Fences.** Each of these is said once, at the slide where it applies:
  - quasi-transversal CCZ versus a physical layer with no correction (slides 16 and 30)
  - the Nezami–Haah classification range $n + k \leq 38$ versus our bound $n \leq 38$ (slide 14)
  - 8-divisibility is a design choice, not part of the conditions (slide 18)
  - the 47-qubit code of Jacinto et al. is not $\mathcal{Q}_{48}$ with one point removed (slide 30).
- **Verbal punctuation.** Each outline frame says which part of six begins and what it does.
- **Ask a question.** Three planned questions, at slides 9, 11 and 21, each with a 7-second wait.
- **Collaborators first.** Ken Brown and Robert Calderbank are credited on the title slide.
- **Ending.** Slide 31 is read as contributions, and the talk ends with a salute to the audience instead of "thank you".

Pacing checkpoints

Reach this point	Clock	If about one minute behind
Outline before slide 7	7:30	On slide 7, show one coordinate face and name the other two.
Outline before slide 14	15:30	Give the history slide in two sentences. Keep the matrix/space distinction and the parity/sign distinction.
Outline before slide 23	26:00	On slide 22, give the formula, the Q48 entry and the census qualification only.
Outline before slide 28	34:00	Give the Reed–Muller argument in two sentences. Name the four stages of the bound without detail.
Slide 31	39:30	Read the contributions and two open questions. Stop at 41:00.

Do not save time by dropping the sign-feasibility distinction or the hypotheses of the lower bound. If a question interrupts a derivation, give a short answer and offer to return to it in the discussion.

Title slide

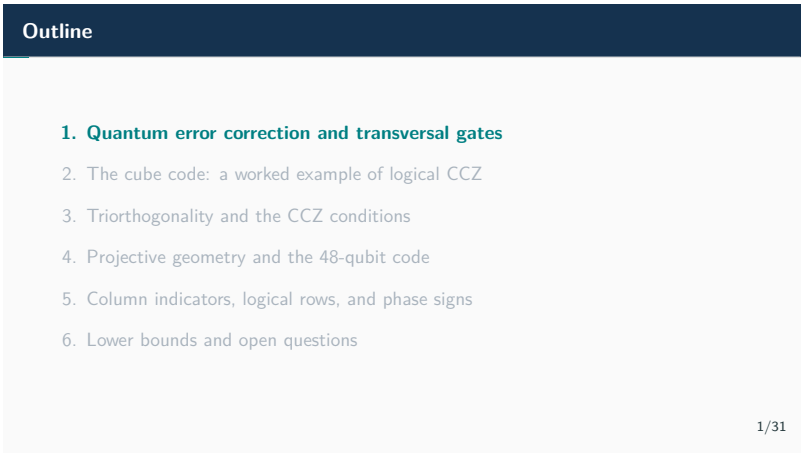


Clock 0:00–0:15 · **Budget** 15 s · **Clicks** 0

On screen: title, Bohan Lu and Ken Brown, arXiv number.

Thank you for the invitation. This is joint work with Ken Brown at Duke, and I also thank Robert Calderbank for pointers on the classical coding theory.

Slide 1. Outline

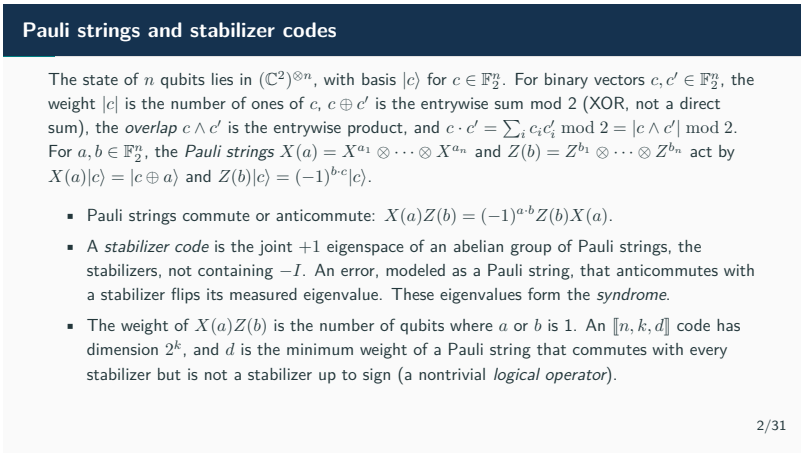


Clock 0:15–1:00 · **Budget** 45 s · **Clicks** 0

On screen: the six parts of the talk, with part 1 highlighted.

By the end of this talk you will be able to take a binary generator matrix and a choice of T or $T^\dagger$ on each qubit, and decide whether the layer of gates acts as a logical CCZ. *(pause)* We start with the quantum background we need, then work through an eight-qubit example in full. After that comes the construction of a larger code, and finally the lower bounds.

Slide 2. Pauli strings and stabilizer codes



Clock 1:00–2:30 · **Budget** 90 s · **Clicks** 3

On screen: the basis states $|c\rangle$, the notation $|c|$, $\oplus$, $\wedge$ and $\cdot$ for binary vectors, and the Pauli strings $X(a)$ and $Z(b)$ with their action on $|c\rangle$.

First, the quantum notation we need. The basis states of n qubits are labelled by binary vectors $c \in \mathbb{F}_2^n$. Four pieces of vector notation. The weight $|c|$ is the number of ones of c. The sum $c \oplus c'$ is entrywise addition mod 2. It is XOR, not a direct sum. The overlap $c \wedge c'$ is the entrywise product, one where both are one. And $c \cdot c' = \sum_i c_i c'_i \bmod 2$ is the dot product, which equals $|c \wedge c'| \bmod 2$. *(pause)* For binary vectors $a, b \in \mathbb{F}_2^n$, the Pauli strings $X(a)$ and $Z(b)$ put an X or a Z on each qubit where a or b is one. $X(a)$ sends $|c\rangle$ to $|c \oplus a\rangle$, and $Z(b)$ multiplies $|c\rangle$ by $(-1)^{b \cdot c}$. So $Z(b)$ gives a minus sign exactly when b and c overlap in an odd number of places.

[click] Pauli strings commute or anticommute. $X(a)Z(b) = (-1)^{a \cdot b} Z(b)X(a)$, so they commute when $a \cdot b$ is even and anticommute when it is odd.

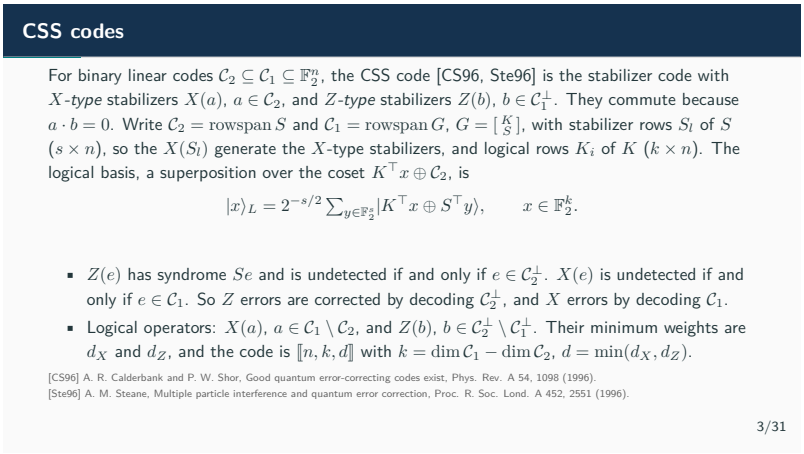
[click] A stabilizer code is the joint $+1$ eigenspace of an abelian group of Pauli strings, the stabilizers, not containing $-I$. An error that anticommutes with a stabilizer flips its measured eigenvalue, and these eigenvalues form the syndrome.

[click] The weight of $X(a)Z(b)$ is the number of qubits where a or b is one. An $[\![n, k, d]\!]$ code has dimension 2^k, and d is the minimum weight of a Pauli string that commutes with every stabilizer but is not itself a stabilizer, up to sign. Those strings are the nontrivial logical operators.

Background (not spoken)

- **Dot product versus overlap.** $c \cdot c'$ is a single bit, $\sum_i c_i c'_i \bmod 2$. The overlap $c \wedge c'$ is a vector. They are related by $c \cdot c' = |c \wedge c'| \bmod 2$. Later slides use the overlap and its integer weight for signed weights, and the dot product only as a sign exponent or a parity.
- **Notation.** $X(a)$ and $Z(b)$ with $a, b \in \mathbb{F}_2^n$ is the notation of Camps-Moreno et al. and of Rengaswamy, Calderbank and coauthors. The Pauli group is generated by $\pm X(a)Z(b)$, with $Y = iXZ$ on a single qubit. For general strings, $X(a)Z(b)$ and $X(a')Z(b')$ commute up to the sign $(-1)^{a \cdot b' + b \cdot a'}$, the symplectic inner product. The talk only needs the case $X(a)$ against $Z(b)$.
- **Why Pauli strings suffice.** I, X, Z and XZ span the 2×2 complex matrices. The Knill–Laflamme conditions are linear in the error, so a code that corrects a set of Pauli errors also corrects every linear combination of them.
- **Why an anticommuting error flips the eigenvalue.** If $P|\psi\rangle = |\psi\rangle$ and $PE = -EP$, then $P(E|\psi\rangle) = -E|\psi\rangle$. So the corrupted state is in the -1 eigenspace of P.
- **Why eigenvalues are ± 1.** Each stabilizer is a Hermitian Pauli string with $P^2 = I$.
- **Code dimension.** With r independent stabilizer generators the code space has dimension 2^{n-r}, so $k = n - r$.
- **Distance.** d is the smallest weight of a Pauli string that commutes with every stabilizer but is not a stabilizer up to phase. These are the nontrivial logical operators.

Slide 3. CSS codes



CSS codes

For binary linear codes $\mathcal{C}_2 \subseteq \mathcal{C}_1 \subseteq \mathbb{F}_2^n$, the CSS code [CS96, Ste96] is the stabilizer code with X-*type* stabilizers $X(a)$, $a \in \mathcal{C}_2$, and Z-*type* stabilizers $Z(b)$, $b \in \mathcal{C}_1^\perp$. They commute because $a \cdot b = 0$. Write $\mathcal{C}_2 = \operatorname{rowspan} S$ and $\mathcal{C}_1 = \operatorname{rowspan} G$, $G = [\begin{smallmatrix} K \\ S \end{smallmatrix}]$, with stabilizer rows S_l of S ($s \times n$), so the $X(S_l)$ generate the X-type stabilizers, and logical rows K_i of K ($k \times n$). The logical basis, a superposition over the coset $K^\top x \oplus \mathcal{C}_2$, is

$$|x\rangle_L = 2^{-s/2} \textstyle\sum_{y \in \mathbb{F}_2^s} |K^\top x \oplus S^\top y\rangle, \qquad x \in \mathbb{F}_2^k.$$

- $Z(e)$ has syndrome Se and is undetected if and only if $e \in \mathcal{C}_2^\perp$. $X(e)$ is undetected if and only if $e \in \mathcal{C}_1$. So Z errors are corrected by decoding $\mathcal{C}_2^\perp$, and X errors by decoding $\mathcal{C}_1$.
- Logical operators: $X(a)$, $a \in \mathcal{C}_1 \setminus \mathcal{C}_2$, and $Z(b)$, $b \in \mathcal{C}_2^\perp \setminus \mathcal{C}_1^\perp$. Their minimum weights are d_X and d_Z, and the code is $[\![n, k, d]\!]$ with $k = \dim \mathcal{C}_1 - \dim \mathcal{C}_2$, $d = \min(d_X, d_Z)$.

[CS96] A. R. Calderbank and P. W. Shor, Good quantum error-correcting codes exist, Phys. Rev. A 54, 1098 (1996).
[Ste96] A. M. Steane, Multiple particle interference and quantum error correction, Proc. R. Soc. Lond. A 452, 2551 (1996).

3/31

Clock 2:30–4:00 · **Budget** 90 s · **Clicks** 2

On screen: $\mathcal{C}_2 \subseteq \mathcal{C}_1$, the stabilizers $X(a)$ and $Z(b)$, the matrices S and K, and the logical basis $|x\rangle_L$.

Our codes are CSS codes, built from two nested binary codes, $\mathcal{C}_2 \subseteq \mathcal{C}_1$. The X-type stabilizers are $X(a)$ for $a \in \mathcal{C}_2$, and the Z-type stabilizers are $Z(b)$ for $b \in \mathcal{C}_1^\perp$. They commute because $a \cdot b = 0$. We write $\mathcal{C}_2$ as the row space of the stabilizer rows S_l of a matrix S, so the $X(S_l)$ generate the X-type stabilizers, and $\mathcal{C}_1$ as the row space of G, the logical rows K_i of a matrix K stacked on S. The logical basis state with label x is the uniform superposition of $K^\top x \oplus S^\top y$ over all y. That is a sum over the coset $K^\top x \oplus \mathcal{C}_2$.

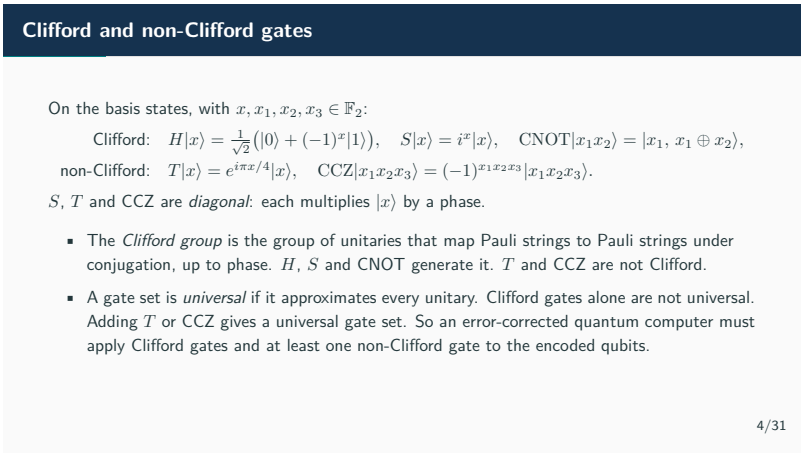
[click] A Z error $Z(e)$ has syndrome Se, and it is undetected exactly when $e \in \mathcal{C}_2^\perp$. An X error $X(e)$ is undetected exactly when $e \in \mathcal{C}_1$. So Z errors are corrected by decoding $\mathcal{C}_2^\perp$, and X errors by decoding $\mathcal{C}_1$.

[click] The logical operators are $X(a)$ with $a \in \mathcal{C}_1 \setminus \mathcal{C}_2$, and $Z(b)$ with $b \in \mathcal{C}_2^\perp \setminus \mathcal{C}_1^\perp$. Their minimum weights are d_X and d_Z, and $d = \min(d_X, d_Z)$.

Background (not spoken)

- **Why $X(a)$ with $a \in \mathcal{C}_2$ fixes $|x\rangle_L$.** $X(a)$ sends $|K^\top x \oplus S^\top y\rangle$ to another term of the same sum, so it only permutes the terms.
- **Why $Z(b)$ with $b \in \mathcal{C}_1^\perp$ fixes $|x\rangle_L$.** $Z(b)|c\rangle = (-1)^{b \cdot c}|c\rangle$, and $b \cdot c = 0$ for every $c \in \mathcal{C}_1$.
- **Why they commute.** $X(a)Z(b) = (-1)^{a \cdot b} Z(b)X(a)$, and $a \cdot b = 0$ because $a \in \mathcal{C}_2 \subseteq \mathcal{C}_1$ and $b \in \mathcal{C}_1^\perp$.
- **Logical Z.** For $b \in \mathcal{C}_2^\perp \setminus \mathcal{C}_1^\perp$, $Z(b)$ acts on $|x\rangle_L$ as the sign $(-1)^{b \cdot (K^\top x)}$, which depends on x. That is why it is a nontrivial logical operator.
- **Counting.** $k = \dim \mathcal{C}_1 - \dim \mathcal{C}_2$. There are s X-type generators and $n - k - s$ Z-type generators, $n - k$ in all.
- **Notation in the literature.** Nielsen and Chuang, Camps-Moreno et al. and our paper write the CSS code of $\mathcal{C}_2 \subseteq \mathcal{C}_1$ as here. Our paper writes the logical state as $K^\top x + S^\top y$. The slides write the same vector as $K^\top x \oplus S^\top y$, with $\oplus$ for addition mod 2, to keep it apart from the integer sums in signed weights.
- **If asked "is every stabilizer code CSS?"** No. In a CSS code every stabilizer generator is all X or all Z. All codes in this talk are CSS.
- **If asked about the Steane code.** It has $\mathcal{C}_2 = \mathcal{C}_1^\perp$ ($\mathcal{C}_1$ is the Hamming code), so the X-type and Z-type generators have the same supports.

Slide 4. Clifford and non-Clifford gates



Clock 4:00–5:15 · **Budget** 75 s · **Clicks** 2

On screen: H, S, CNOT, T and CCZ, each written by its action on basis states, and the definition of a diagonal gate.

Now gates. I write every gate by what it does to a basis state $|x\rangle$, with x a bit. *(pause)* H sends $|x\rangle$ to $|0\rangle + (-1)^x|1\rangle$, divided by $\sqrt{2}$. S multiplies $|x\rangle$ by i^x. CNOT adds the first bit to the second. T multiplies $|x\rangle$ by $e^{i\pi x/4}$. And CCZ multiplies $|x_1x_2x_3\rangle$ by $(-1)^{x_1x_2x_3}$, so it puts a minus sign on $|111\rangle$ and nothing else. S, T and CCZ are diagonal: each multiplies a basis state by a phase.

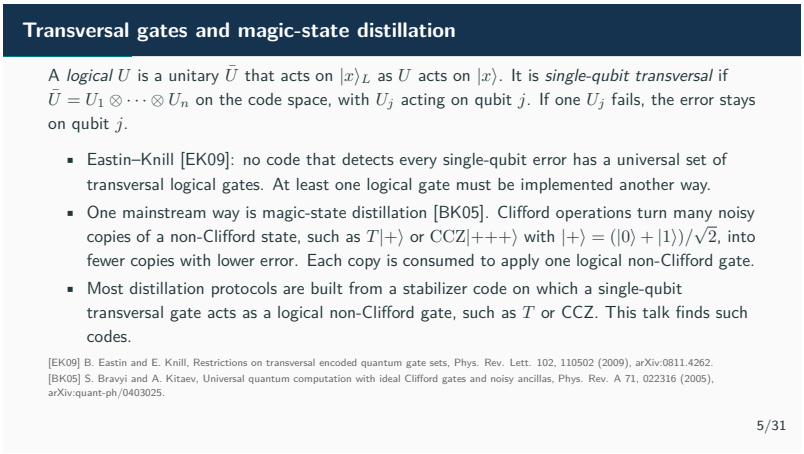
[click] The Clifford group is the group of unitaries that map Pauli strings to Pauli strings under conjugation, up to phase. H, S and CNOT generate it. That is the first row. T and CCZ, in the second row, are not Clifford.

[click] A gate set is universal if it approximates every unitary. Clifford gates alone are not universal. A Clifford circuit that starts from a basis state can be simulated efficiently on a classical computer. Adding T, or adding CCZ, gives a universal gate set. *(pause)* So an error-corrected quantum computer must apply Clifford gates and at least one non-Clifford gate to the encoded qubits.

Background (not spoken)

- **Clifford group.** The unitaries U with $UPU^\dagger$ a Pauli string, up to phase, for every Pauli string P. For example, $HXH = Z$, $SXS^\dagger = Y$ with $Y = iXZ$, and CNOT maps $X \otimes I$ to $X \otimes X$.
- **Why T is not Clifford.** $TXT^\dagger = (X + Y)/\sqrt{2}$, which is not a Pauli string up to phase. For CCZ, conjugating X on the first qubit gives X times a controlled-Z on the other two, which is not a Pauli string.
- **Why Clifford circuits are not universal.** By the Gottesman–Knill theorem, a Clifford circuit on stabilizer states with Pauli measurements can be simulated efficiently on a classical computer.
- **Universality.** Clifford + T generates a dense subgroup of the unitary group up to phase. CCZ with H gives Toffoli, and Clifford + CCZ is also universal.
- **CNOT convention.** $|x_1, x_1 \oplus x_2\rangle$: the first qubit is the control.
- **If asked "why CCZ rather than T?"** The standard Clifford + T circuit for CCZ uses seven T gates. Campbell and Howard distill CCZ states directly, which can lower the cost.

Slide 5. Transversal gates and magic-state distillation



Clock 5:15–6:15 · **Budget** 60 s · **Clicks** 3

On screen: the definition of a single-qubit transversal gate.

How do we apply a gate to encoded qubits? A logical U is a unitary that acts on the logical basis state $|x\rangle_L$ as U acts on $|x\rangle$. The simplest way to get one is a single-qubit transversal gate: a tensor product $U_1 \otimes \cdots \otimes U_n$, one gate on each physical qubit. If one of these gates fails, the error stays on that qubit.

[click] Eastin and Knill proved that no code that detects every single-qubit error has a universal set of transversal logical gates. So at least one logical gate has to be implemented another way.

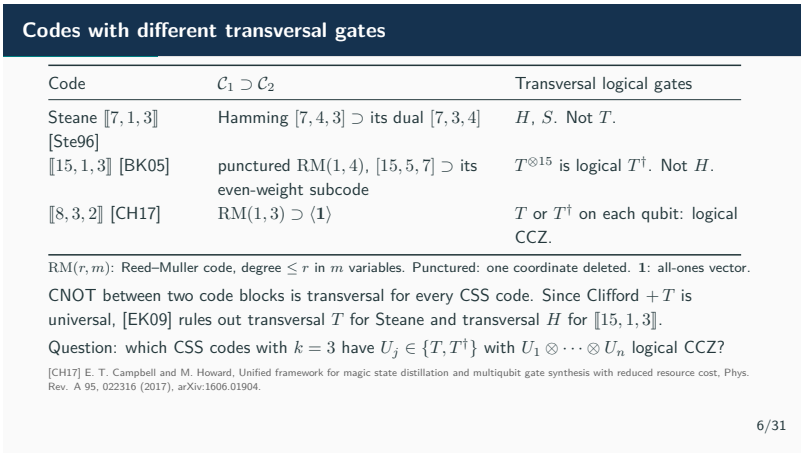
[click] One mainstream way is magic-state distillation, introduced by Bravyi and Kitaev. Clifford operations turn many noisy copies of a non-Clifford state, such as $T|+\rangle$, into fewer copies with lower error. Each copy is then consumed to apply one non-Clifford gate.

[click] Most distillation protocols are built from a stabilizer code on which a transversal gate acts as a logical T or CCZ. *(pause)* So the question becomes a question about codes: which stabilizer codes have a transversal non-Clifford gate? The next slide gives three examples.

Background (not spoken)

- **The idea behind Eastin–Knill.** The transversal logical gates form a group. If the code detects every single-qubit error, a single-qubit unitary close to the identity acts trivially on the code space. So the group of transversal logical gates is finite, and a finite group cannot be dense in the unitary group.
- **Magic states.** $|T\rangle = T|+\rangle$ and $|\mathrm{CCZ}\rangle = \mathrm{CCZ}|+++\rangle$, with $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. Gate teleportation consumes one such state, together with Clifford gates and measurement, to apply T or CCZ.
- **If asked how the code is used.** In the 15-to-1 protocol of Bravyi and Kitaev, fifteen noisy T states apply $T^{\otimes 15}$ on the $[\![15, 1, 3]\!]$ code. Measuring the stabilizers detects every pattern of one or two faulty states. To leading order the output error is $35p^3$ for input error p.
- **If asked about "single-qubit".** The gates U_j may differ from qubit to qubit. The cube code uses T on some qubits and $T^\dagger$ on others.

Slide 6. Codes with different transversal gates



Codes with different transversal gates

Code	$\mathcal{C}_1 \supset \mathcal{C}_2$	Transversal logical gates
Steane $[\![7,1,3]\!]$ [Ste96]	Hamming $[7,4,3] \supset$ its dual $[7,3,4]$	H, S. Not T.
$[\![15,1,3]\!]$ [BK05]	punctured $\mathrm{RM}(1,4)$, $[15,5,7] \supset$ its even-weight subcode	$T^{\otimes 15}$ is logical $T^\dagger$. Not H.
$[\![8,3,2]\!]$ [CH17]	$\mathrm{RM}(1,3) \supset \langle \mathbf{1} \rangle$	T or $T^\dagger$ on each qubit: logical CCZ.

$\mathrm{RM}(r,m)$: Reed–Muller code, degree $\leq r$ in m variables. Punctured: one coordinate deleted. $\mathbf{1}$: all-ones vector.

CNOT between two code blocks is transversal for every CSS code. Since Clifford $+T$ is universal, [EK09] rules out transversal T for Steane and transversal H for $[\![15,1,3]\!]$.

Question: which CSS codes with $k = 3$ have $U_j \in \{T, T^\dagger\}$ with $U_1 \otimes \cdots \otimes U_n$ logical CCZ?

[CH17] E. T. Campbell and M. Howard, Unified framework for magic state distillation and multiqubit gate synthesis with reduced resource cost, Phys. Rev. A 95, 022316 (2017), arXiv:1606.01904.

6/31

Clock 6:15–7:30 · **Budget** 75 s · **Clicks** 2

On screen: a table of three codes with their $\mathcal{C}_1 \supset \mathcal{C}_2$ and transversal logical gates, and a note on $\mathrm{RM}(r,m)$ and **1**.

Three codes show the tradeoff. Two notations first: $\mathrm{RM}(r,m)$ is the Reed–Muller code of polynomials of degree at most r in m variables, and **1** is the all-ones vector. The Steane code takes $\mathcal{C}_1$ to be the Hamming code and $\mathcal{C}_2$ its dual. It has transversal H and S, but not T. The fifteen-qubit code takes $\mathcal{C}_1$ to be punctured $\mathrm{RM}(1,4)$, that is, with one coordinate deleted, and $\mathcal{C}_2$ its even-weight subcode. T on every qubit acts as logical $T^\dagger$, but H is not transversal. The cube code, $[\![8,3,2]\!]$, uses $\mathrm{RM}(1,3)$ over the all-ones vector. There T or $T^\dagger$ on each qubit acts as logical CCZ on the three encoded qubits.

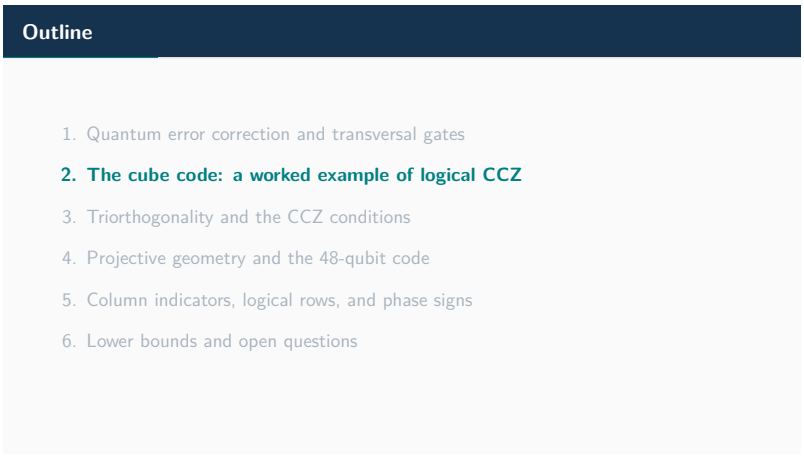
[click] CNOT between two code blocks is transversal for every CSS code. So the Steane code has the whole Clifford group transversally. Since Clifford + T is universal, Eastin–Knill rules out transversal T on the Steane code and transversal H on the fifteen-qubit code.

[click] Here is the question for this talk. *(pause)* For which CSS codes with three logical qubits can we choose T or $T^\dagger$ on each qubit, so that the layer acts exactly as logical CCZ? The cube code does it with distance two. We want distance three.

Background (not spoken)

- **Steane, S gate.** The even coset $\mathcal{C}_2$ has weights 0 and 4, and the odd coset has weights 3 and 7. S on every qubit multiplies $|c\rangle$ by $i^{|c|}$, giving 1 on the even coset and $i^3 = -i$ on the odd coset. So $S^{\otimes 7}$ is logical $S^\dagger$, and $(S^\dagger)^{\otimes 7}$ is logical S.
- **Steane, H gate.** H on every qubit exchanges X-type and Z-type generators. This preserves the code because $\mathcal{C}_2 = \mathcal{C}_1^\perp$, so both types have the same supports.
- **Fifteen-qubit code, T gate.** $\mathcal{C}_2$ has weights 0 and 8, giving phase 1. The coset $\mathbf{1} \oplus \mathcal{C}_2$ has weights 7 and 15, both $\equiv 7 \pmod 8$, giving $e^{7i\pi/4} = e^{-i\pi/4}$. So $T^{\otimes 15}$ is logical $T^\dagger$. Its distances are $d_X = 7$ and $d_Z = 3$.
- **Cube code.** $d_X = 4$ and $d_Z = 2$, as the next slides show.
- **If asked "why not use the fifteen-qubit code?"** It gives T on one logical qubit. We want CCZ on three logical qubits at distance 3.
- **If asked who introduced the fifteen-qubit code for distillation.** Bravyi and Kitaev used it for T distillation [BK05]. It is the quantum Reed–Muller code.

Outline before slide 7 (part 2 of 6)

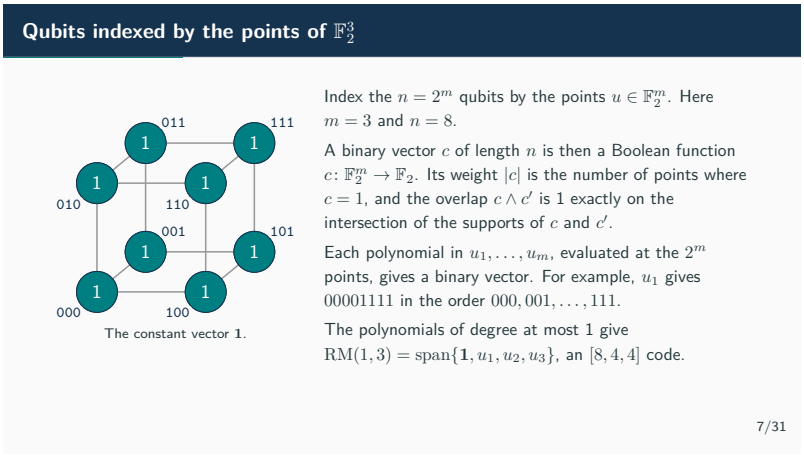


Clock 7:30–7:40 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 2 highlighted.

That is the background. Part two is the eight-qubit cube, worked out in full: its encoding, its logical phase, and the overlap conditions that give CCZ.

Slide 7. Qubits indexed by the points of $\mathbb{F}_2^3$



Clock 7:40–8:45 · **Budget** 65 s · **Clicks** 4

On screen: the cube $\mathbb{F}_2^3$. Binary vectors as Boolean functions, weight, and overlap.

Index the eight qubits by the points of $\mathbb{F}_2^3$, the vertices of this cube. A binary vector of length eight is then a Boolean function on the cube, and its weight is the number of points where it is one. The overlap of two vectors is one exactly on the intersection of their supports, so it is a vertex, an edge or a face when the supports are faces.

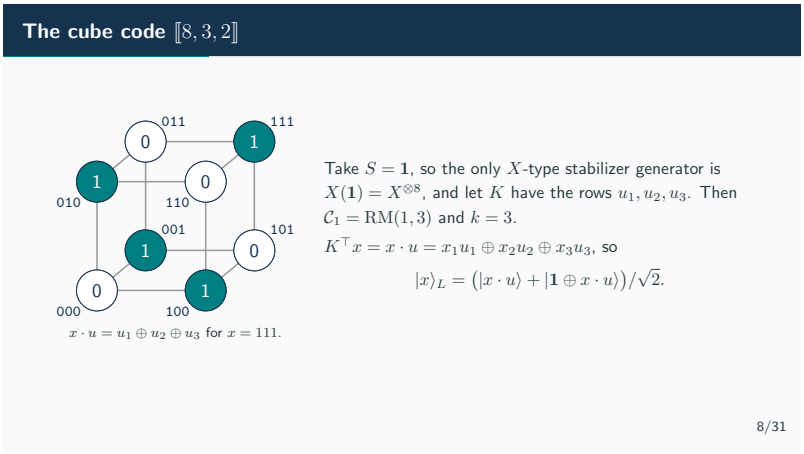
[click] Each polynomial in u_1, u_2, u_3 gives a binary vector by evaluation at the points. Here is u_1. It is one on the face where the first coordinate is one.

[click] u_2 is the face in the second direction,

[click] u_3 the face in the third direction,

[click] and the constant function **1** is one at all eight vertices. These four functions are the polynomials of degree at most one, so they span RM(1, 3), an $[8, 4, 4]$ code. Two faces meet in an edge, and three faces meet in a vertex.

Slide 8. The cube code $[\![8, 3, 2]\!]$



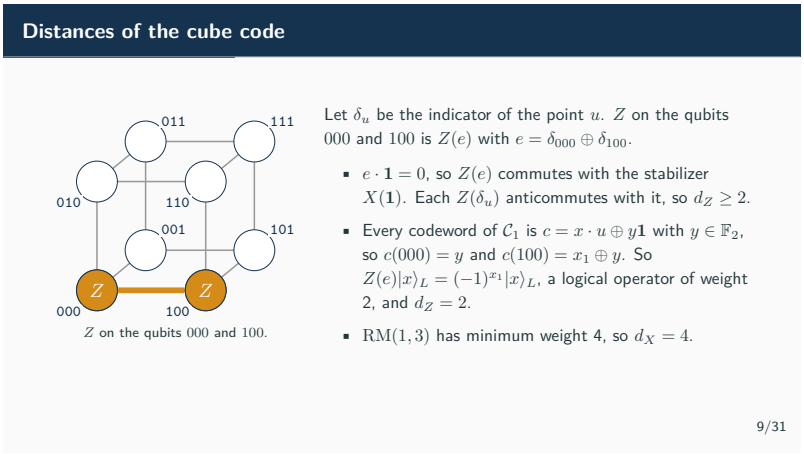
Clock 8:45–9:30 · **Budget** 45 s · **Clicks** 1

On screen: $S = \mathbf{1}$, $K = (u_1, u_2, u_3)$, and the logical basis state of the cube code. The click shows $x \cdot u$ for $x = 111$ on the cube.

Now the code. Take $S = \mathbf{1}$, so the only X-type stabilizer generator is $X(\mathbf{1})$, the X on all eight qubits. Take K to have the three coordinate functions u_1, u_2, u_3 as rows. Then $\mathcal{C}_1 = \mathrm{RM}(1, 3)$ and $k = 3$. The label x gives $K^\mathsf{T}x = x \cdot u = x_1u_1 \oplus x_2u_2 \oplus x_3u_3$. So each logical basis state is the superposition of $x \cdot u$ and its complement.

[click] For $x = 111$, $x \cdot u$ is $u_1 \oplus u_2 \oplus u_3$, shown on the cube. This is the label that CCZ multiplies by minus one.

Slide 9. Distances of the cube code



Clock 9:30–10:30 · **Budget** 60 s · **Clicks** 3

On screen: the edge from 000 to 100 with Z on both ends.

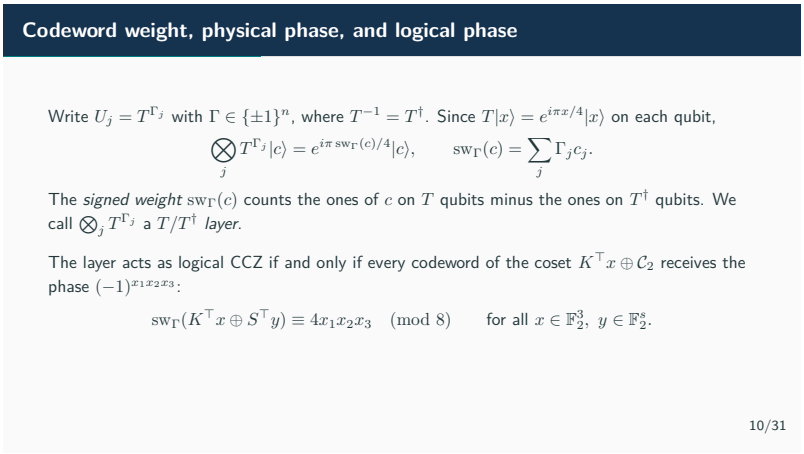
Before adding gates, let us see what this code protects. *(ask, then wait 7 seconds)* How many Z errors does it take to change a logical state of this code without being detected? *(Fallback: two, and here is an example.)* Put Z on the qubits 000 and 100. That is $Z(e)$, where $e = \delta_{000} \oplus \delta_{100}$ and δ_u is the indicator of the point u.

[click] $e \cdot \mathbf{1} = 0$, so $Z(e)$ commutes with the stabilizer $X(\mathbf{1})$. A single $Z(\delta_u)$ anticommutes with it, so $d_Z \geq 2$.

[click] Every codeword of $\mathcal{C}_1$ is $c = x \cdot u \oplus y\mathbf{1}$, with y a bit. Its values at 000 and 100 are y and $x_1 \oplus y$. So $Z(e)|x\rangle_L = (-1)^{x_1}|x\rangle_L$. That is a logical operator of weight two, so $d_Z = 2$.

[click] On the X side, RM(1, 3) has minimum weight four, so $d_X = 4$. *(pause)* So the cube code has distance two. We now show that T or $T^\dagger$ on each qubit acts as logical CCZ.

Slide 10. Codeword weight, physical phase, and logical phase



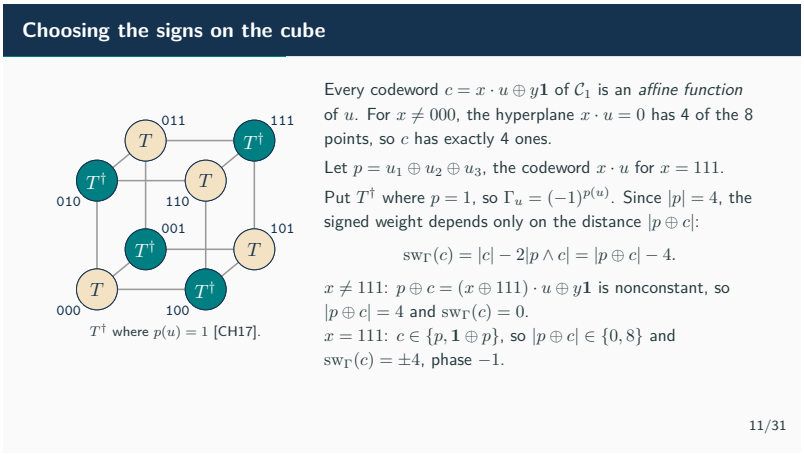
Clock 10:30–11:30 · **Budget** 60 s · **Clicks** 1

On screen: the phase of a layer of T and $T^\dagger$ on a basis state, and the signed weight sw_Γ.

Why does a layer of T gates act by a phase? Write the gate on qubit j as T^{Γ_j}, with $\Gamma_j = \pm 1$, so T^{-1} is $T^\dagger$. Each T^{Γ_j} multiplies $|c_j\rangle$ by $e^{i\pi\Gamma_j c_j/4}$. We call the product a $T/T^\dagger$ layer. It multiplies the basis state $|c\rangle$ by $e^{i\pi \mathrm{sw}_\Gamma(c)/4}$. Here $\mathrm{sw}_\Gamma(c) = \sum_j \Gamma_j c_j$ is the signed weight: the number of ones of c on T qubits minus the number on $T^\dagger$ qubits. It is an integer, not a parity.

[click] The layer is diagonal, so it multiplies each term of a logical state by that term's own phase. It acts as a logical gate only if all terms of a logical state receive the same phase. For CCZ that phase is $(-1)^{x_1x_2x_3}$. In terms of signed weights, $\mathrm{sw}_\Gamma(K^\top x \oplus S^\top y) \equiv 4x_1x_2x_3 \pmod 8$ for all x and y. This congruence is the gate condition, and the rest of the talk rewrites it.

Slide 11. Choosing the signs on the cube



Clock 11:30–13:00 · **Budget** 90 s · **Clicks** 2

On screen: codewords as affine functions with four ones, and $p = u_1 \oplus u_2 \oplus u_3$.

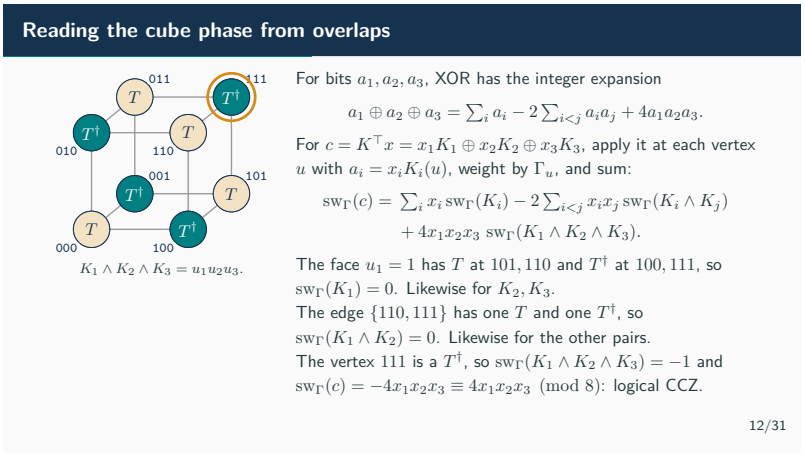
Every codeword $c = x \cdot u \oplus y\mathbf{1}$ is an affine function of the point u. For $x \neq 000$, the hyperplane $x \cdot u = 0$ has four of the eight points, so c has exactly four ones. Let $p = u_1 \oplus u_2 \oplus u_3$, the codeword of the label 111.

(ask, then wait 7 seconds) The label 111 must get a minus sign and every other label a plus sign. Where would you put the $T^\dagger$ gates? *(Fallback: on the four points where p is one.)*

[click] Put $T^\dagger$ where $p = 1$ and T where $p = 0$. Then $\mathrm{sw}_\Gamma(c) = |c| - 2|p \wedge c|$. Since p has weight four, this equals $|p \oplus c| - 4$. So the signed weight depends only on the distance from c to p.

[click] If $x \neq 111$, then $p \oplus c$ is a nonconstant affine function. Its weight is four, so $\mathrm{sw}_\Gamma = 0$ and the phase is one. If $x = 111$, the codeword is p or its complement, at distances 0 and 8. The signed weights are -4 and $+4$, and both give the phase -1. So both terms of the logical state $|111\rangle_L$ get the same sign, and the layer is exactly CCZ.

Slide 12. Reading the cube phase from overlaps



Clock 13:00–14:45 · **Budget** 105 s · **Clicks** 3

On screen: the $T/T^\dagger$ pattern on the cube, the integer expansion of $a_1 \oplus a_2 \oplus a_3$, and $\mathrm{sw}_\Gamma(c)$ in terms of overlaps.

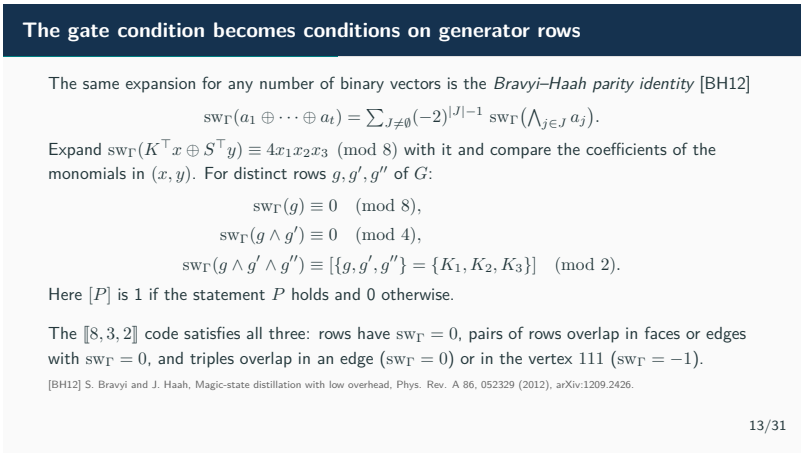
There is a second way to see this, and it is the one that generalizes. For three bits, XOR has an integer expansion: $a_1 \oplus a_2 \oplus a_3 = \sum_i a_i - 2\sum_{i<j} a_ia_j + 4a_1a_2a_3$. Take $c = x_1K_1 \oplus x_2K_2 \oplus x_3K_3$, where K_i is row i of K. Apply the expansion at each vertex u with $a_i = x_iK_i(u)$, weight by Γ_u, and sum. That gives $\mathrm{sw}_\Gamma(c)$ in terms of the signed weights of the rows K_i, of their pairwise overlaps, and of the triple overlap. *(pause)*

[click] $K_1 = u_1$ is the face $u_1 = 1$. It has T at 101 and 110, and $T^\dagger$ at 100 and 111. So $\mathrm{sw}_\Gamma(K_1) = 0$, and the same holds for K_2 and K_3.

[click] $K_1 \wedge K_2$ is the edge $\{110, 111\}$. That is one T and one $T^\dagger$, so $\mathrm{sw}_\Gamma(K_1 \wedge K_2) = 0$, and likewise for the other pairs.

[click] $K_1 \wedge K_2 \wedge K_3$ is the vertex 111, which carries $T^\dagger$. So $\mathrm{sw}_\Gamma(K_1 \wedge K_2 \wedge K_3) = -1$, and $\mathrm{sw}_\Gamma(c) = -4x_1x_2x_3 \equiv 4x_1x_2x_3 \pmod 8$. That is CCZ, read off a face, an edge and a vertex. The stabilizer row **1** adds terms of the same kind, a face and an edge, and they vanish too.

Slide 13. The gate condition becomes conditions on generator rows



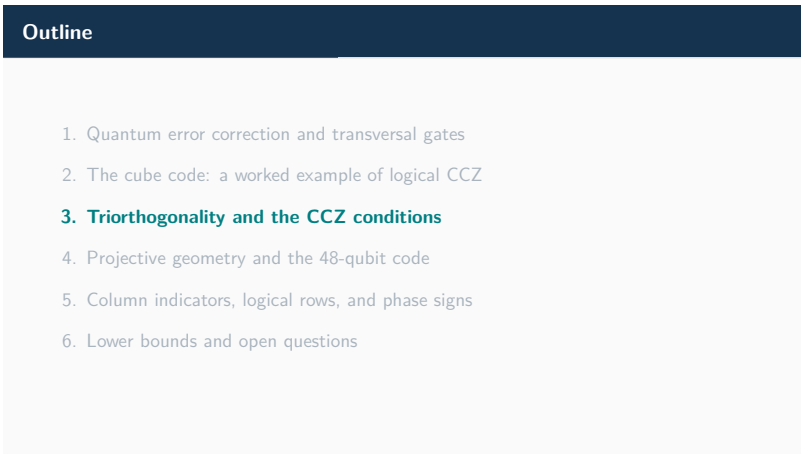
Clock 14:45–15:30 · **Budget** 45 s · **Clicks** 1

On screen: the Bravyi–Haah parity identity, and the three congruences for single rows, pairs and triples.

The same expansion holds for any number of vectors. This is the Bravyi–Haah parity identity: an overlap of t vectors has coefficient $(-2)^{t-1}$. Expand the gate condition this way and compare coefficients. For distinct rows g, g', g'' of G, single rows need $\mathrm{sw}_\Gamma(g) \equiv 0 \pmod 8$. Pair overlaps carry -2, so they need $\mathrm{sw}_\Gamma(g \wedge g') \equiv 0 \pmod 4$. Triple overlaps carry 4, so only their parity matters, and it must be odd exactly for the triple K_1, K_2, K_3. Overlaps of four or more rows carry a multiple of eight and drop out.

[click] The cube code satisfies all three. These are the conditions we impose on larger codes.

Outline before slide 14 (part 3 of 6)

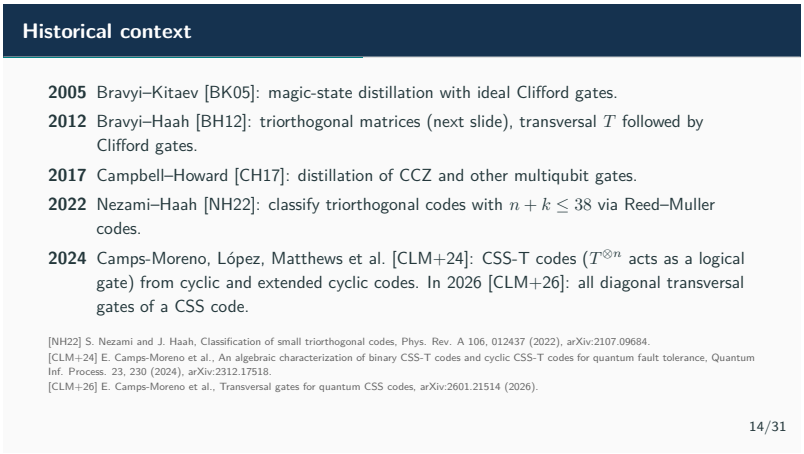


Clock 15:30–15:40 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 3 highlighted.

Part three. The cube gave exact signed conditions. Now I relate them to triorthogonality, and separate the parity conditions from the choice of signs.

Slide 14. Historical context

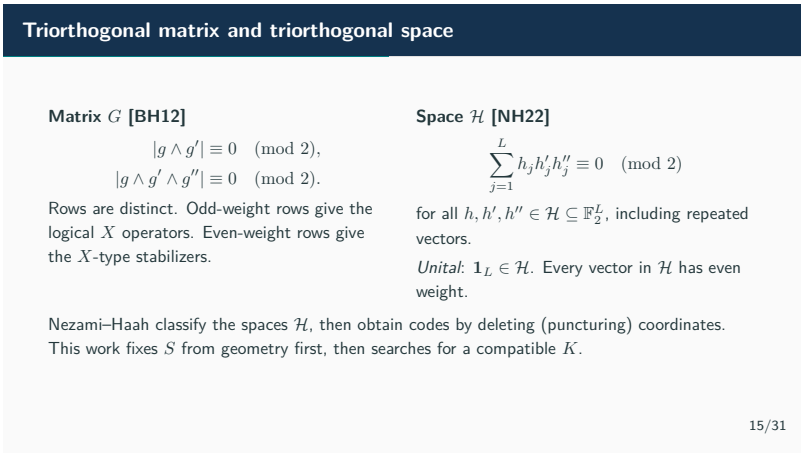


Clock 15:40–16:15 · **Budget** 35 s · **Clicks** 0

On screen: a timeline from 2005 to 2026.

Briefly, the history. Bravyi and Kitaev introduced magic-state distillation. Bravyi and Haah wrote transversal T as parity conditions on binary rows, triorthogonal matrices. Campbell and Howard gave the CCZ version, which we use. Nezami and Haah classified triorthogonal codes with $n + k \leq 38$. That is a range for T codes, and it is a different statement from our bound $n \leq 38$. And Camps-Moreno, López, Matthews and coauthors study CSS-T codes, CSS codes where T on every qubit acts as a logical gate. In 2026 they computed all diagonal transversal gates of a CSS code.

Slide 15. Triorthogonal matrix and triorthogonal space

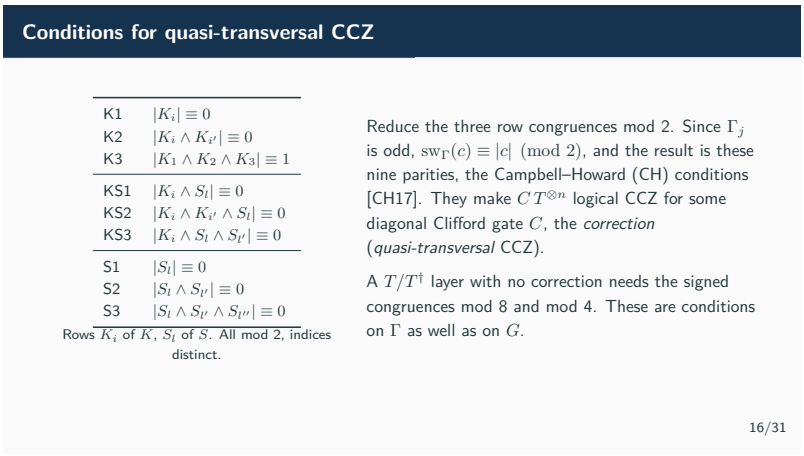


Clock 16:15–17:30 · **Budget** 75 s · **Clicks** 0

On screen: the Bravyi–Haah matrix conditions beside the Nezami–Haah space condition.

There are two related definitions, and they are easy to confuse. On the left is the triorthogonal matrix of Bravyi and Haah. Pair overlaps and triple overlaps of distinct rows are even. Odd-weight rows give the logical X operators, and even-weight rows give the X-type stabilizers. On the right is the triorthogonal space $\mathcal{H}$ of Nezami and Haah. The triple sum $\sum_j h_j h'_j h''_j$ vanishes mod 2 for all $h, h', h'' \in \mathcal{H}$, including repeated vectors. Taking $h = h' = h''$ shows every vector has even weight. Unital means the all-ones vector is in the space. So the row span of a triorthogonal matrix with odd rows is not a triorthogonal space. *(pause)* Nezami and Haah classify the spaces and obtain codes by deleting, or puncturing, coordinates. We go the other way. We fix S from geometry first, then search for a compatible K. And CCZ needs one odd triple, K_1, K_2, K_3, which neither definition allows. That is why we need the generalized conditions on the next slide.

Slide 16. Conditions for quasi-transversal CCZ



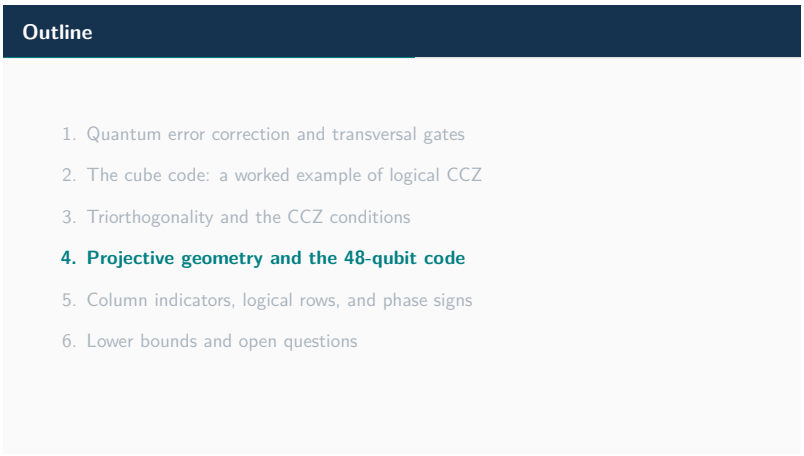
Clock 17:30–19:00 · **Budget** 90 s · **Clicks** 1

On screen: the nine parity conditions K1–K3, KS1–KS3 and S1–S3.

Reduce the three signed conditions mod 2. Each Γ_j is odd, so a signed weight has the same parity as the ordinary weight. What remains are these nine parity conditions of Campbell and Howard, the CH conditions. K1 to K3 involve only logical rows, KS1 to KS3 mix logical and stabilizer rows, and S1 to S3 involve only stabilizer rows. K3 is the only odd condition, and it selects CCZ. These nine conditions make T on every qubit, followed by some diagonal Clifford gate C, act as logical CCZ. C is called the correction, and this is quasi-transversal CCZ.

[click] Here is a distinction I want to keep clear. *(pause)* A physical layer of T and $T^\dagger$ with no correction needs more, namely the signed congruences mod 8 and mod 4. Those are conditions on the signs Γ as well as on the matrix. A code can satisfy all nine parities and still have no valid choice of signs. We will see an example at length 47.

Outline before slide 17 (part 4 of 6)

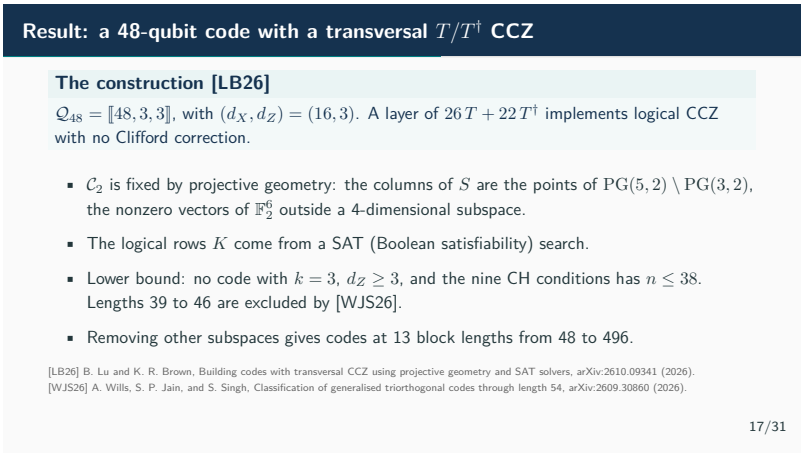


Clock 19:00–19:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 4 highlighted.

Part four, the construction. We now build the 48-qubit code, starting from the geometry that fixes its stabilizers.

Slide 17. Result: a 48-qubit code with a transversal $T/T^\dagger$ CCZ

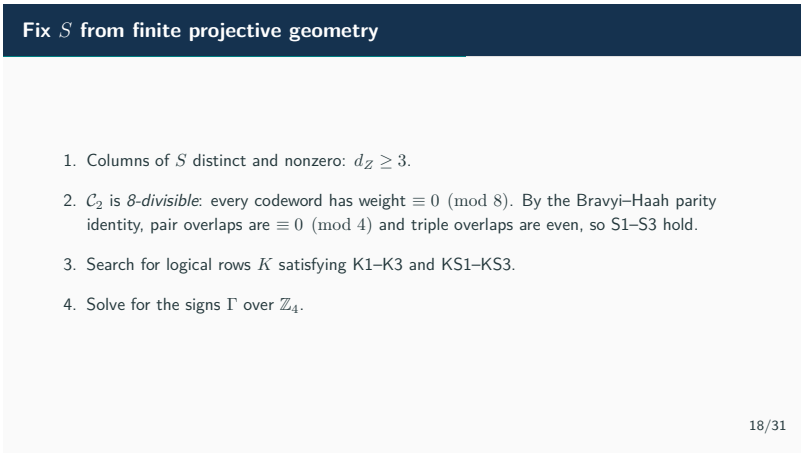


Clock 19:10–20:30 · **Budget** 80 s · **Clicks** 0

On screen: $\mathcal{Q}_{48} = [\![48, 3, 3]\!]$ with $(d_X, d_Z) = (16, 3)$, and four bullets on the construction and the bounds.

Here is the result. $\mathcal{Q}_{48}$ is a $[\![48, 3, 3]\!]$ code, with $d_X = 16$ and $d_Z = 3$. A layer of 26 T gates and 22 $T^\dagger$ gates implements logical CCZ, with no Clifford correction. It is built in three steps. Geometry fixes the stabilizers: the columns of S are the points of $\mathrm{PG}(5,2) \setminus \mathrm{PG}(3,2)$, the nonzero vectors of $\mathbb{F}_2^6$ outside a 4-dimensional subspace. A SAT search, a Boolean satisfiability solver, finds the logical rows K. And linear algebra over $\mathbb{Z}_4$ finds the signs. *(pause)* For the lower bound, our theorem says that no code with three logical qubits, $d_Z \geq 3$, and the nine CH conditions has $n \leq 38$. Lengths 39 to 46 are excluded by the classification of Wills, Jain and Singh. Removing other subspaces gives codes at 13 block lengths from 48 to 496.

Slide 18. Fix S from finite projective geometry



Clock 20:30–21:30 · **Budget** 60 s · **Clicks** 3

On screen: step 1, distinct nonzero columns of S give $d_Z \geq 3$.

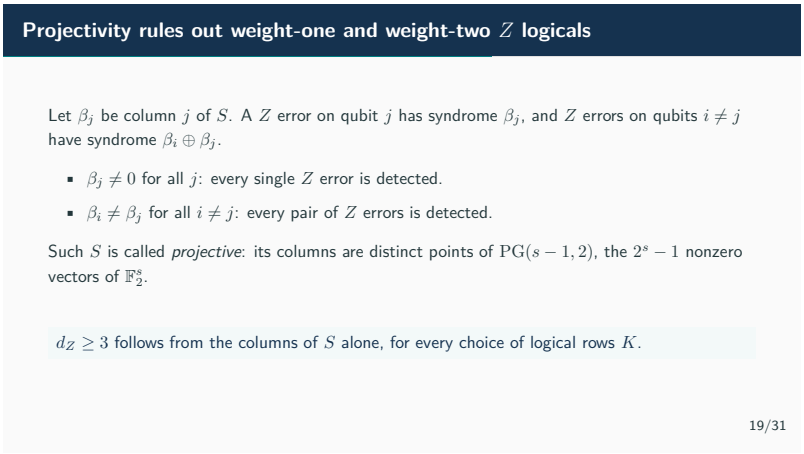
The construction has four steps. First, choose S with distinct nonzero columns. That alone gives $d_Z \geq 3$, as the next slide shows.

[click] Second, require $\mathcal{C}_2$ to be 8-divisible: every codeword has weight divisible by 8. By the Bravyi–Haah parity identity, pair overlaps are then zero mod 4 and triple overlaps are even, so S1 to S3 hold. This divisibility is our design choice. The CH conditions do not require it.

[click] Third, search for logical rows K that satisfy K1 to K3 and KS1 to KS3. This is the only search.

[click] Fourth, solve for the signs Γ over $\mathbb{Z}_4$. That is a linear-algebra test.

Slide 19. Projectivity rules out weight-one and weight-two Z logicals

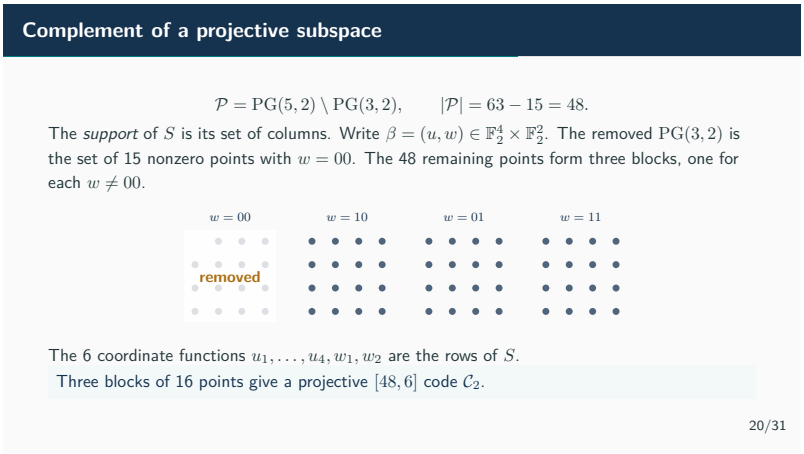


Clock 21:30–22:30 · **Budget** 60 s · **Clicks** 0

On screen: the syndrome of a Z error is a column β_j of S, and distinct nonzero columns detect one or two Z errors.

Why distinct nonzero columns? Let β_j be column j of S. A Z error on qubit j has syndrome β_j, and Z errors on qubits $i \neq j$ have syndrome $\beta_i \oplus \beta_j$. If no column is zero, every single Z error is detected. If the columns are distinct, $\beta_i \oplus \beta_j \neq 0$, so every pair of Z errors is detected too. So $d_Z \geq 3$. *(pause)* Such an S is called projective: its columns are distinct points of $\mathrm{PG}(s-1,2)$, the $2^s - 1$ nonzero vectors of $\mathbb{F}_2^s$. This is the parity-check argument for a classical code of distance three. It does not involve the logical rows, and it covers d_Z only. We compute d_X for the final code.

Slide 20. Complement of a projective subspace



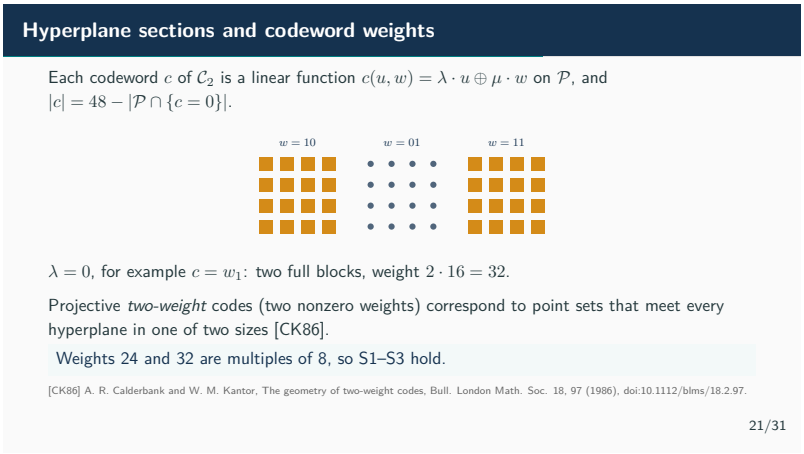
Clock 22:30–23:30 · **Budget** 60 s · **Clicks** 1

On screen: $\mathcal{P} = \mathrm{PG}(5,2) \setminus \mathrm{PG}(3,2)$ with $63 - 15 = 48$ points, drawn as four blocks indexed by w.

Take the 63 points of PG(5, 2) and remove a PG(3, 2), which has 15 points. That leaves 48. The support of S is its set of columns. Write a point as $\beta = (u, w)$, with $u \in \mathbb{F}_2^4$ and $w \in \mathbb{F}_2^2$. The removed subspace is the 15 nonzero points with $w = 00$. In the picture, the four blocks are the values of w. Inside each block the rows are u_1u_2 and the columns are u_3u_4, in Gray order.

[click] Remove the block $w = 00$. The 48 remaining points form three blocks, one for each $w \neq 00$. The six coordinate functions $u_1, \ldots, u_4, w_1, w_2$, evaluated on the 48 points, are the rows of S. Three blocks of 16 points give a projective $[48, 6]$ code, $\mathcal{C}_2$. We keep this picture for the weights, the logical rows and the signs.

Slide 21. Hyperplane sections and codeword weights



Clock 23:30–25:00 · **Budget** 90 s · **Clicks** 1

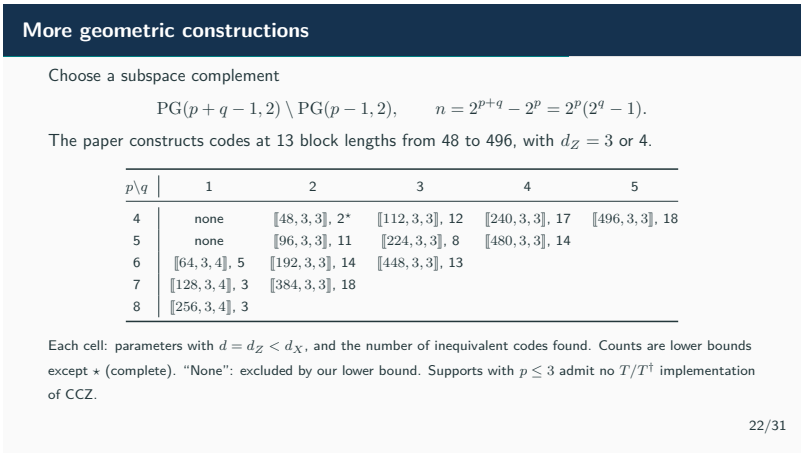
On screen: a codeword of $\mathcal{C}_2$ as a linear function on the 48 points, with u_1 marked in gold.

Now the weights. Each codeword of $\mathcal{C}_2$ is a linear function $c(u, w) = \lambda \cdot u \oplus \mu \cdot w$ on the 48 points. Its weight is 48 minus the number of points on the hyperplane where it vanishes. If $\lambda \neq 0$, for example $c = u_1$, the gold squares show that it is one on eight points of each block. So the weight is $3 \cdot 8 = 24$.

(ask, then wait 7 seconds) What is the weight of w_1? *(Fallback: click and read it off.)*

[click] w_1 is one on two full blocks, so its weight is $2 \cdot 16 = 32$. These are the only two cases. So the nonzero weights are 24 and 32, both multiples of 8, and S1 to S3 hold. *(pause)* A projective code with two nonzero weights is a two-weight code. Calderbank and Kantor show these codes correspond to point sets that meet every hyperplane in one of two sizes.

Slide 22. More geometric constructions



More geometric constructions

Choose a subspace complement

$$\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2), \qquad n = 2^{p+q} - 2^p = 2^p(2^q-1).$$

The paper constructs codes at 13 block lengths from 48 to 496, with $d_Z = 3$ or 4.

$p\backslash q$	1	2	3	4	5
4	none	$[\![48,3,3]\!]$, 2*	$[\![112,3,3]\!]$, 12	$[\![240,3,3]\!]$, 17	$[\![496,3,3]\!]$, 18
5	none	$[\![96,3,3]\!]$, 11	$[\![224,3,3]\!]$, 8	$[\![480,3,3]\!]$, 14	
6	$[\![64,3,4]\!]$, 5	$[\![192,3,3]\!]$, 14	$[\![448,3,3]\!]$, 13		
7	$[\![128,3,4]\!]$, 3	$[\![384,3,3]\!]$, 18			
8	$[\![256,3,4]\!]$, 3				

Each cell: parameters with $d = d_Z < d_X$, and the number of inequivalent codes found. Counts are lower bounds except $\star$ (complete). "None": excluded by our lower bound. Supports with $p \leq 3$ admit no $T/T^\dagger$ implementation of CCZ.

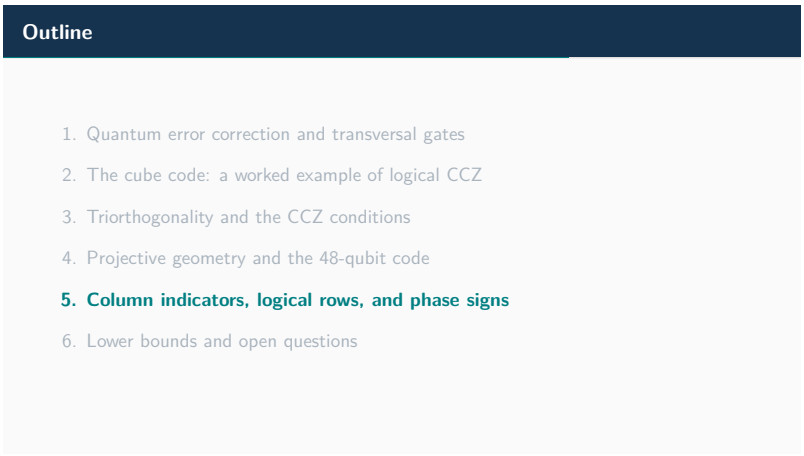
22/31

Clock 25:00–26:00 · **Budget** 60 s · **Clicks** 0

On screen: $n = 2^p(2^q-1)$ for $\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2)$, and a table of codes found for $p = 4$ to 8 and $q = 1$ to 5.

The same idea works for any subspace complement, $\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2)$. The length is $n = 2^p(2^q-1)$. $\mathcal{Q}_{48}$ is the case $p = 4$, $q = 2$. The paper constructs codes at 13 block lengths from 48 to 496, with $d_Z = 3$ or 4. Each cell gives the parameters and the number of inequivalent codes we found. For example, $p = 6$ and $q = 1$ gives 64 qubits with $d_Z = 4$. *(pause)* Two qualifications. Only the starred entry at 48 is a complete census for that support. The other counts are lower bounds. The entries marked "none" are excluded by our lower bound, and supports with $p \leq 3$ admit no $T/T^\dagger$ implementation of CCZ. This is a family of finite constructions, not an asymptotic statement.

Outline before slide 23 (part 5 of 6)

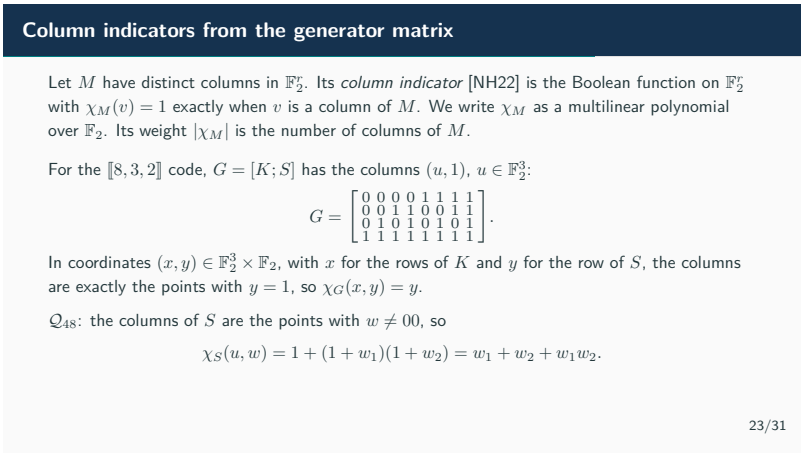


Clock 26:00–26:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 5 highlighted.

Part five. Geometry has fixed the stabilizers. Now we find the logical rows using column indicators, and then solve for the signs.

Slide 23. Column indicators from the generator matrix



Clock 26:10–27:30 · **Budget** 80 s · **Clicks** 2

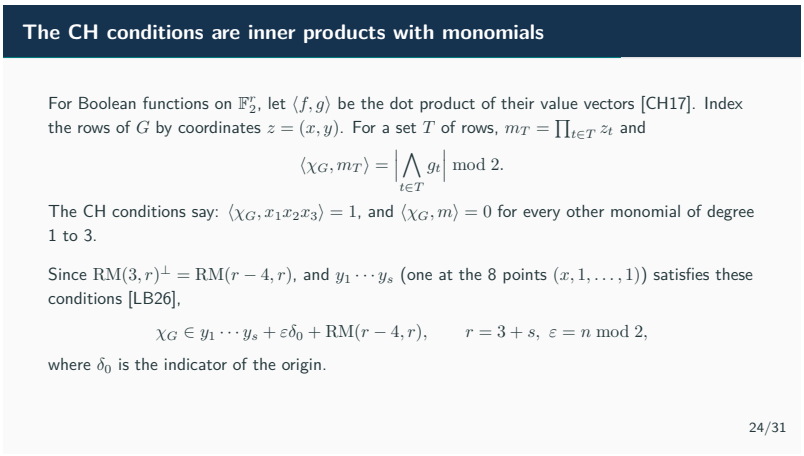
On screen: the definition of the column indicator χ_M.

To find the logical rows we switch from rows to columns. Let M have distinct columns in $\mathbb{F}_2^r$. Its column indicator, as in Nezami and Haah, is the Boolean function χ_M on $\mathbb{F}_2^r$ with $\chi_M(v) = 1$ exactly when v is a column of M. We write it as a multilinear polynomial over $\mathbb{F}_2$. Its weight is the number of columns of M.

[click] For the cube, $G = [K; S]$ has the columns $(u, 1)$ for every $u \in \mathbb{F}_2^3$. In coordinates (x, y), with x for the rows of K and y for the row of S, every x occurs with $y = 1$, and no column has $y = 0$. So $\chi_G(x, y) = y$.

[click] For $\mathcal{Q}_{48}$, the columns of S are the points with $w \neq 00$. So $\chi_S(u, w) = 1 + (1 + w_1)(1 + w_2) = w_1 + w_2 + w_1 w_2$. It has degree two and does not depend on u.

Slide 24. The CH conditions are inner products with monomials



Clock 27:30–29:00 · **Budget** 90 s · **Clicks** 2

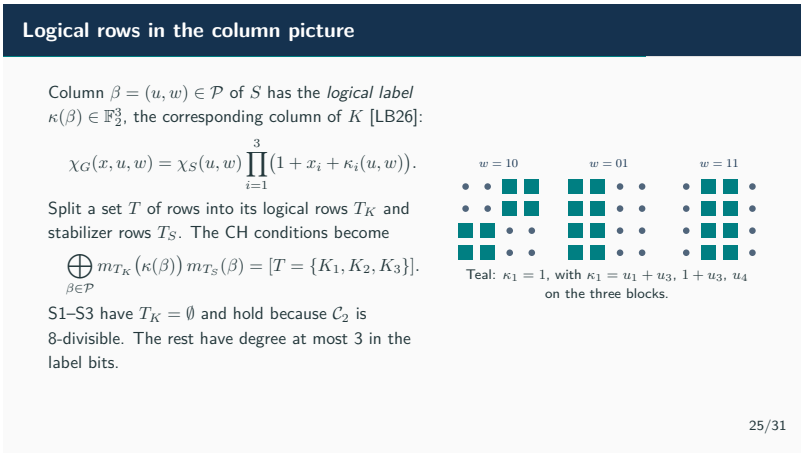
On screen: the inner product $\langle f, g\rangle$ and the identity $\langle \chi_G, m_T\rangle = |\bigwedge_{t\in T} g_t| \bmod 2$.

Why columns? Because overlaps become inner products. For Boolean functions on $\mathbb{F}_2^r$, $\langle f, g\rangle = \sum_v f(v)g(v) \bmod 2$, the dot product of their value vectors. Index the rows of G by coordinates $z = (x, y)$. For a set T of rows, let $m_T = \prod_{t\in T} z_t$ be the monomial. Then $\langle \chi_G, m_T\rangle$ is the weight of the overlap of those rows, mod 2. Summing over the columns is summing over the points where χ_G is one.

[click] So the CH conditions say this: $\langle \chi_G, x_1x_2x_3\rangle = 1$, and $\langle \chi_G, m\rangle = 0$ for every other monomial of degree one to three.

[click] The dual of $\mathrm{RM}(3, r)$ is $\mathrm{RM}(r-4, r)$. The product $y_1 \cdots y_s$ satisfies these conditions. It is one at the eight points $(x, 1, \dots, 1)$, and among these monomials only $x_1x_2x_3$ has odd inner product with it. So every valid χ_G lies in $y_1 \cdots y_s + \varepsilon\delta_0 + \mathrm{RM}(r-4, r)$, where $r = 3 + s$. Here δ_0 is the indicator of the origin and $\varepsilon = n \bmod 2$. That coset statement is a lemma in our paper.

Slide 25. Logical rows in the column picture



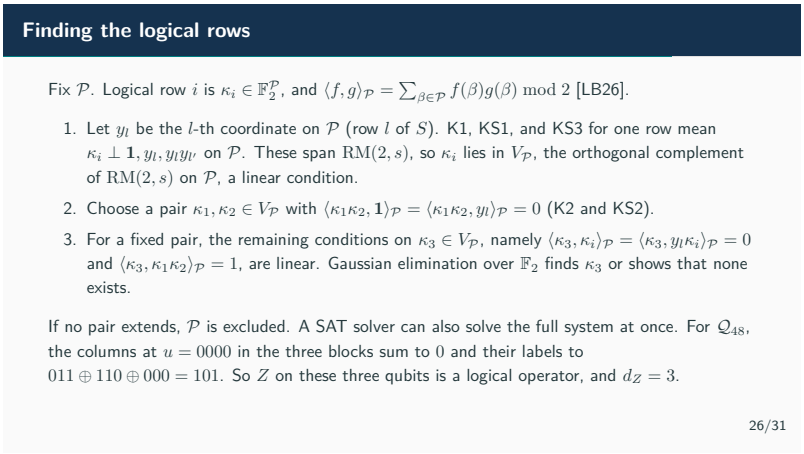
Clock 29:00–30:30 · **Budget** 90 s · **Clicks** 1

On screen: the logical label $\kappa(\beta)$, the factorization of χ_G, and the CH conditions as sums over the 48 points.

For $\mathcal{Q}_{48}$, S is fixed, so the unknowns are the logical rows. In the column picture, each column β of S has a logical label $\kappa(\beta) \in \mathbb{F}_2^3$, the corresponding column of K. Then $\chi_G(x, u, w) = \chi_S(u, w) \prod_{i=1}^{3}(1+x_i+\kappa_i(u, w))$, which forces x to equal κ at each point. Split a set T of rows into its logical rows T_K and stabilizer rows T_S. The CH conditions become a sum over the 48 points, which must equal one exactly for the logical triple. S1 to S3 have no logical rows, and they hold because $\mathcal{C}_2$ is 8-divisible. The rest are equations of degree at most three in the 144 label bits.

[click] Here is the first logical row of $\mathcal{Q}_{48}$ on the support. κ_1 is $u_1 + u_3$ on the first block, $1 + u_3$ on the second, and u_4 on the third. It has eight ones in each block.

Slide 26. Finding the logical rows



Clock 30:30–32:30 · **Budget** 120 s · **Clicks** 3

On screen: step 1 of Algorithm 1, $\kappa_i \in V_{\mathcal{P}}$, the orthogonal complement of $\mathrm{RM}(2, s)$ on $\mathcal{P}$.

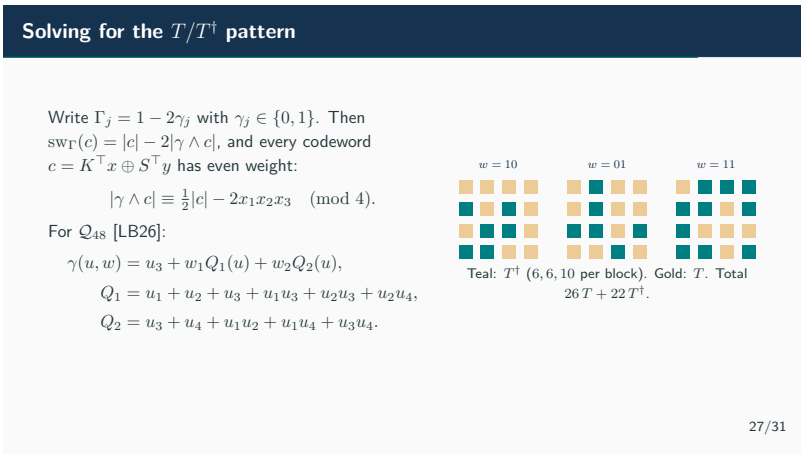
How do we solve these equations? Fix the support $\mathcal{P}$. Logical row i is a function κ_i on $\mathcal{P}$, and the inner product $\langle f, g\rangle_{\mathcal{P}}$ is summed over $\mathcal{P}$. This is Algorithm 1 in the paper. Let y_l be the l-th coordinate on $\mathcal{P}$, which is row l of S. Step one: K1, KS1 and KS3 for a single row say that κ_i is orthogonal to $\mathbf{1}$, to each y_l, and to each product $y_l y_{l'}$. Those functions span $\mathrm{RM}(2, s)$. So κ_i lies in $V_{\mathcal{P}}$, the orthogonal complement of $\mathrm{RM}(2, s)$ on $\mathcal{P}$, which is a linear condition.

[click] Step two: choose a pair $\kappa_1, \kappa_2 \in V_{\mathcal{P}}$ whose product is orthogonal to $\mathbf{1}$ and to every y_l. Those are the conditions K2 and KS2.

[click] Step three: once the pair is fixed, every remaining condition on κ_3 is linear. It must be orthogonal to κ_i and to $y_l\kappa_i$, and have inner product one with $\kappa_1\kappa_2$. Gaussian elimination over $\mathbb{F}_2$ finds κ_3 or proves that none exists.

[click] If no pair extends, the support is excluded. A SAT solver can also solve the whole system at once. *(pause)* For $\mathcal{Q}_{48}$, take the three columns at $u = 0000$ in the three blocks. They sum to zero, so Z on these three qubits commutes with every X-type stabilizer. Their labels sum to $011 \oplus 110 \oplus 000 = 101$, which is not zero. So Z on these three qubits is a logical operator, and $d_Z = 3$.

Slide 27. Solving for the $T/T^\dagger$ pattern



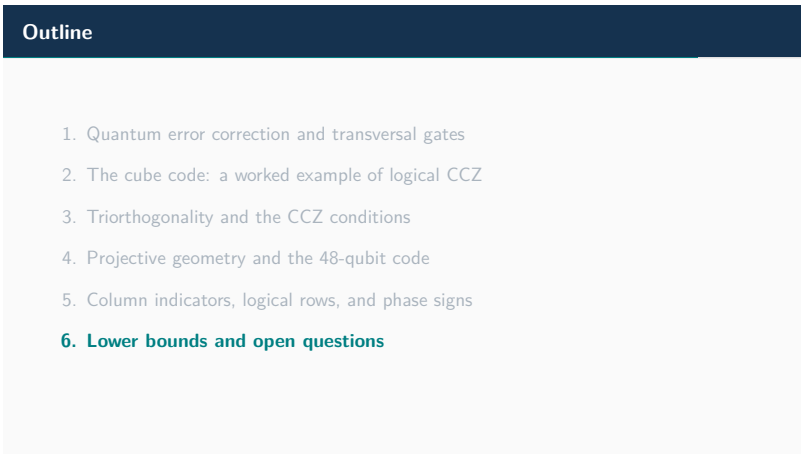
Clock 32:30–34:00 · **Budget** 90 s · **Clicks** 1

On screen: $\Gamma_j = 1 - 2\gamma_j$ and the congruence $|\gamma \wedge c| \equiv \frac{1}{2}|c| - 2x_1x_2x_3 \pmod 4$.

The last step is the signs. Write $\Gamma_j = 1 - 2\gamma_j$, with γ_j a bit that is one where we put $T^\dagger$. Then $\mathrm{sw}_\Gamma(c) = |c| - 2|\gamma \wedge c|$, an ordinary integer count. Every codeword has even weight, so we can divide the gate condition by two. The weight of the overlap must satisfy $|\gamma \wedge c| \equiv \frac{1}{2}|c| - 2x_1x_2x_3 \pmod 4$. That is linear in γ over $\mathbb{Z}_4$, with the extra requirement that γ is binary. Elimination over $\mathbb{Z}_4$ decides whether a solution exists.

[click] For $\mathcal{Q}_{48}$, here is the solution from the paper. $\gamma(u,w) = u_3 + w_1Q_1(u) + w_2Q_2(u)$, with two quadratic functions Q_1 and Q_2. In the picture, teal is $T^\dagger$, with 6, 6 and 10 in the three blocks, 22 in all. Gold is T, 26 in all. These are coordinate positions, not a hardware layout.

Outline before slide 28 (part 6 of 6)

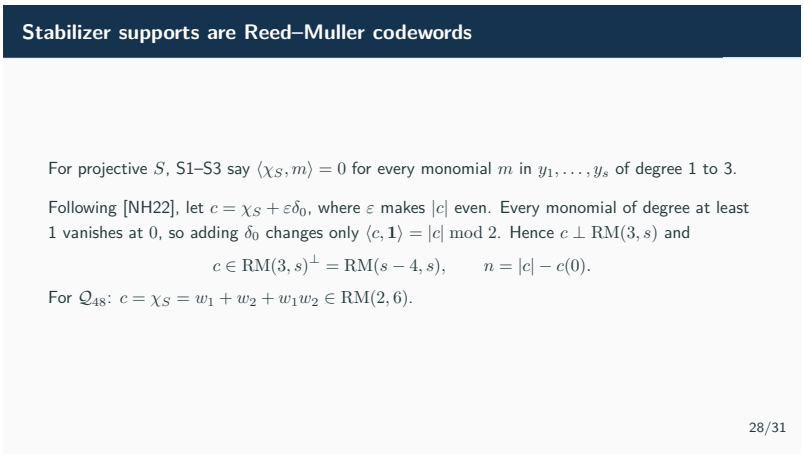


Clock 34:00–34:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 6 highlighted.

Part six, the last part. The construction is complete. Now the same Boolean view explains the lower bound and what remains open.

Slide 28. Stabilizer supports are Reed–Muller codewords

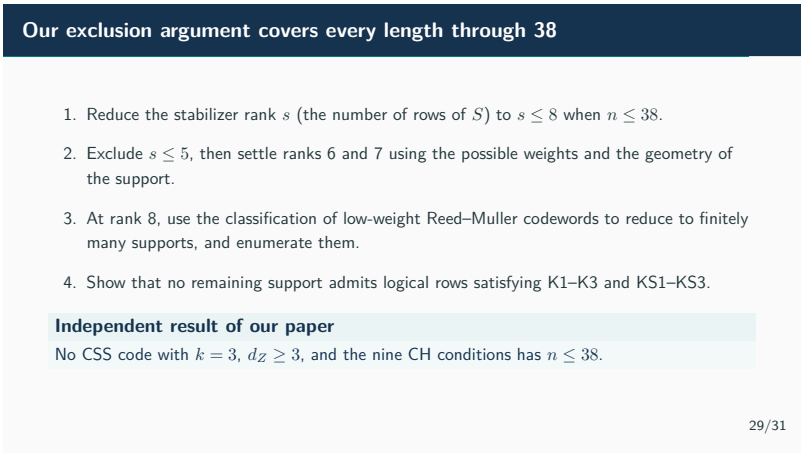


Clock 34:10–36:00 · **Budget** 110 s · **Clicks** 0

On screen: S1–S3 as $\langle \chi_S, m \rangle = 0$ for monomials of degree 1 to 3, and $c = \chi_S + \varepsilon\delta_0 \in \mathrm{RM}(s-4, s)$.

Why can no short code work? Take a projective S. In the column picture, S1 to S3 say that χ_S has inner product zero with every monomial of degree one to three. Following Nezami and Haah, let $c = \chi_S + \varepsilon\delta_0$, with ε chosen to make the weight of c even. Every monomial of positive degree vanishes at the origin. So adding δ_0 changes only the inner product with $\mathbf{1}$, which is the parity of the weight. *(pause)* Hence $c \perp \mathrm{RM}(3, s)$, and $c \in \mathrm{RM}(s-4, s)$. The block length is $n = |c| - c(0)$. For $\mathcal{Q}_{48}$, $c = w_1 + w_2 + w_1 w_2$, a codeword of $\mathrm{RM}(2, 6)$. So a short code needs a low-weight codeword of a Reed–Muller code. Low-weight codewords of Reed–Muller codes are classified, and that is what makes the search below length 39 finite.

Slide 29. Our exclusion argument covers every length through 38

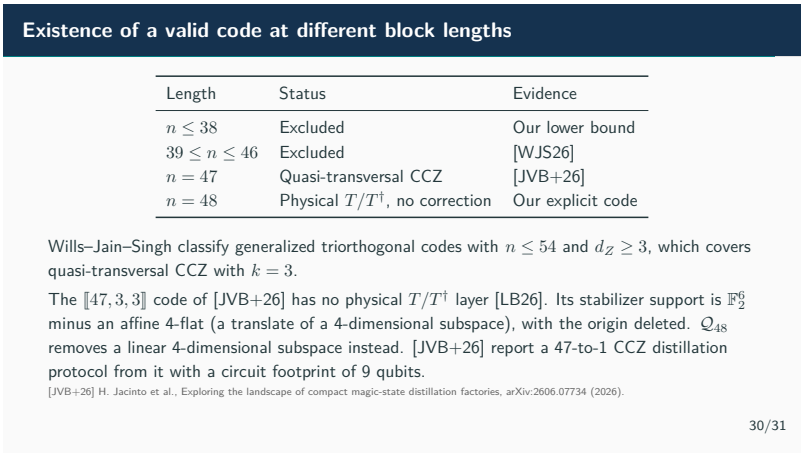


Clock 36:00–37:30 · **Budget** 90 s · **Clicks** 0

On screen: four proof stages, and the theorem: no CSS code with $k = 3$, $d_Z \geq 3$ and the nine CH conditions has $n \leq 38$.

The proof that nothing works through length 38 has four stages. First, reduce the stabilizer rank s, the number of rows of S, to $s \leq 8$ when $n \leq 38$. Second, exclude $s \leq 5$, then settle ranks 6 and 7 using the weight distribution and the geometry of the support. Third, at rank 8, use the classification of low-weight Reed–Muller codewords to reduce to finitely many supports, and enumerate them. Fourth, show that no remaining support admits logical rows satisfying K1 to K3 and KS1 to KS3. *(pause)* The result is our theorem. No CSS code with $k = 3$, $d_Z \geq 3$ and the nine CH conditions has $n \leq 38$. Two remarks. The rank-8 coverage and the logical-row exclusions are complete, not sampled. And because the theorem assumes only the nine parity conditions, it rules out quasi-transversal codes as well as physical layers.

Slide 30. Existence of a valid code at different block lengths



Existence of a valid code at different block lengths

Length	Status	Evidence
$n \leq 38$	Excluded	Our lower bound
$39 \leq n \leq 46$	Excluded	[WJS26]
$n = 47$	Quasi-transversal CCZ	[JVB+26]
$n = 48$	Physical $T/T^\dagger$, no correction	Our explicit code

Wills–Jain–Singh classify generalized triorthogonal codes with $n \leq 54$ and $d_Z \geq 3$, which covers quasi-transversal CCZ with $k = 3$.

The $[\![47, 3, 3]\!]$ code of [JVB+26] has no physical $T/T^\dagger$ layer [LB26]. Its stabilizer support is $\mathbb{F}_2^6$ minus an affine 4-flat (a translate of a 4-dimensional subspace), with the origin deleted. $\mathcal{Q}_{48}$ removes a linear 4-dimensional subspace instead. [JVB+26] report a 47-to-1 CCZ distillation protocol from it with a circuit footprint of 9 qubits.

[JVB+26] H. Jacinto et al., Exploring the landscape of compact magic-state distillation factories, arXiv:2606.07734 (2026).

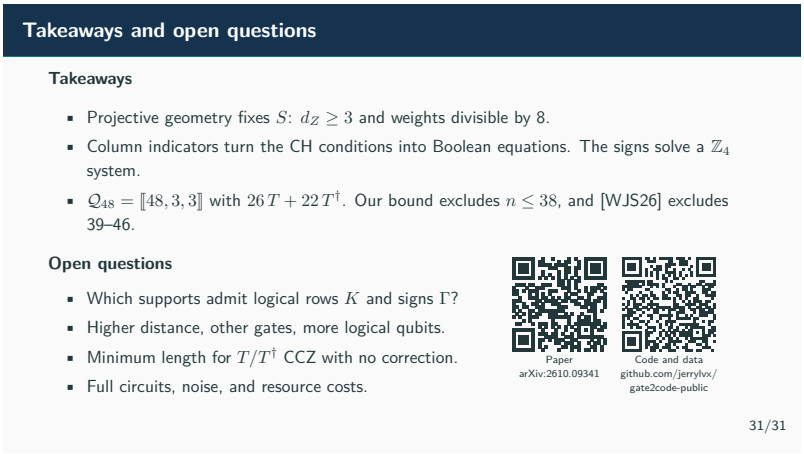
30/31

Clock 37:30–39:30 · **Budget** 120 s · **Clicks** 0

On screen: a table of lengths, status and evidence, and three sentences on the $[\![47, 3, 3]\!]$ code.

Here is the situation by length. Up to 38 is excluded by our bound. 39 to 46 is excluded by Wills, Jain and Singh, who classify generalized triorthogonal codes up to length 54 with $d_Z \geq 3$. That classification covers quasi-transversal CCZ with three logical qubits. At 47 there is a quasi-transversal CCZ code, the $[\![47, 3, 3]\!]$ code of Jacinto and coauthors. At 48 we have $\mathcal{Q}_{48}$, with a physical layer and no correction. *(pause)* Now the distinction between 47 and 48. The 47-qubit code satisfies the nine parities, but it has no physical layer of T and $T^\dagger$. We show this in our paper. So at 47 a Clifford correction is needed. That statement is about this specific code. It does not show that 48 is the minimum length for a physical layer, and it does not show that the 47-qubit code is unique. It is also not $\mathcal{Q}_{48}$ with one point removed. Its stabilizer support is $\mathbb{F}_2^6$ minus an affine 4-flat, a translate of a 4-dimensional subspace, with the origin deleted. $\mathcal{Q}_{48}$ removes a linear 4-dimensional subspace. The stabilizer weights differ as well: 16, 24 and 32 for the 47-qubit code, and 23, 24, 31 and 32 for any puncture of $\mathcal{Q}_{48}$. Finally, Jacinto and coauthors report a 47-to-1 CCZ distillation protocol from their code with circuit footprint of 9 qubits. That number is theirs, and it is about the circuit, not the code length.

Slide 31. Takeaways and open questions



Clock 39:30–41:00 · **Budget** 90 s · **Clicks** 0

On screen: three takeaways, four open questions, and QR codes for the paper and the code.

Here is what we contributed. *(pause)* Projective geometry fixes S, giving $d_Z \geq 3$ and weights divisible by eight. Column indicators turn the CH conditions into Boolean equations for the logical rows, and the signs solve a linear system over $\mathbb{Z}_4$. Together these give $\mathcal{Q}_{48}$, a $[\![48, 3, 3]\!]$ code in which 26 T and 22 $T^\dagger$ gates act as logical CCZ. Our bound excludes lengths up to 38, and Wills, Jain and Singh exclude 39 to 46. In one sentence: geometry fixes the stabilizers, a search finds the logical rows, and linear algebra over $\mathbb{Z}_4$ finds the signs. *(pause)* Some open questions. Which supports admit logical rows and signs? Can we reach higher distance, other gates, or more logical qubits? What is the minimum length for a CCZ layer of T and $T^\dagger$ with no correction? And what do full circuits cost once noise is included? If you work on divisible codes, or on completing a code from a fixed support, I would like to hear your ideas. The QR codes link to the paper and the code. *(salute, then stop)* It has been a pleasure to present this work to algebraists and coding theorists here at Virginia Tech. I look forward to your questions.

After the talk

Leave slide 31 on screen during questions. Prepared answers to likely technical questions are in questions.md.